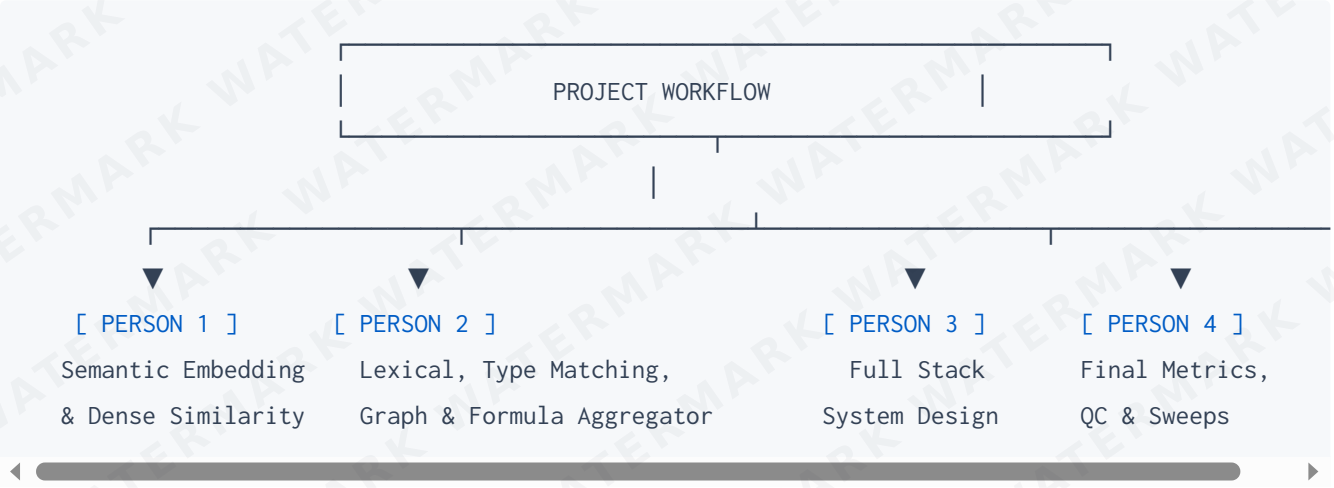
Project Work Division and Review Preparation Guide

AI Natural Language to SQL Query System with Schema Linking and Reduction

This document divides the entire project work evenly across **4 team members** according to specialized technical domains. Each role contains assigned files, core concepts, review speaking scripts, and anticipated viva questions with answers.



Person 1: Semantic Embedding and Dense Representation Subsystem

Role Summary

Responsible for the **dense semantic embedding and vector similarity component** (S_{sem}) of the schema linking algorithm. Focused on bridging the vocabulary gap between natural-language user queries and database identifier names using localized sentence-transformer representations.

Assigned Code and Assets

- schema_linker.py (`compute_semantic_similarity`, `get_sentence_transformer`, dense vector encoding, similarity caching)
- models/ (Local weights caching and offline persistence for `all-MiniLM-L6-v2`)

Key Technical Concepts and Formulations

1. **Semantic Component in Composite Formula**:

$$S_{\text{sem}}(c, q) = \cos(\mathbf{e}_q, \mathbf{e}_c) = \frac{\mathbf{e}_q \cdot \mathbf{e}_c}{\|\mathbf{e}_q\| \|\mathbf{e}_c\|}$$

- Where $\mathbf{e}_q$ is the dense embedding vector of the natural query q, and $\mathbf{e}_c$ is the embedding vector of column c augmented with table context.

2. **Metadata Context Augmentation**: Constructing descriptive schema representations prior to encoding:
    - Example: `Table Student column Attendance representing student attendance percentage`
    - This contextualization ensures embeddings capture column domain semantics beyond raw token names.
3. **Local Embedding Architecture**:
    - Model: `all-MiniLM-L6-v2` (384-dimensional dense vector space).
    - Fast offline inference with local caching in `models/` directory to eliminate external API dependency and latency overhead during schema linking.
4. **Vocabulary Gap Resolution**:
    - Resolving indirect phrasing and semantic synonyms (e.g., matching *"compensation"* or *"earnings"* to `Salary`, *"scores"* to `Marks`, *"who teaches"* to `Faculty`).

Person 2: Lexical Analysis, Type-Matching, Relational Graph and Formula Aggregator

Role Summary

Responsible for the **lexical token matching (S_{lex}), inferred data-type heuristics (S_{type}), foreign-key relational graph propagation (S_{rel}), and the final composite scoring and schema reduction aggregator**.

Assigned Code and Assets

- schema_linker.py (`compute_lexical_similarity`, `compute_type_match_score`, `propagate_relationship_boost`, `link_and_reduce_schema`, `split_identifier`)

Key Technical Concepts and Formulations

1. **Composite Relevance Scoring Function**:

$$\text{Score}(c, q) = w_{\text{lex}} \cdot S_{\text{lex}} + w_{\text{sem}} \cdot S_{\text{sem}} + w_{\text{type}} \cdot S_{\text{type}} + w_{\text{rel}} \cdot S_{\text{rel}}$$

- Configurable default weights: $w_{\text{lex}} = 0.40, w_{\text{sem}} = 0.35, w_{\text{type}} = 0.15, w_{\text{rel}} = 0.10$.

2. **Lexical Matching Algorithms**:
    - Identifier token splitting: Splitting camelCase, PascalCase, and snake_case (`StudentName` $\rightarrow$ `['student', 'name']`).
    - Token overlap, containment Jaccard scoring, and sub-word prefix matching (e.g., query token `attend` matching `Attendance`).
3. **Inferred Data-Type Heuristics (S_{type})**:
    - Rule-based detection matching quantitative intent (`average`, `highest`, `salary`, `>`, `<`, `min`, `max`, `sum`) to numerical types (`INTEGER`, `REAL`), and categorical queries (`name`, `department`) to `TEXT` types.
4. **Relational Graph Propagation (S_{rel})**:
    - Graph-based foreign-key propagation that identifies structural bridge tables. When both `Student` and `Course` receive high relevance, the graph engine boosts `Enrollment` (`StudentID`, `CourseID`) to ensure valid JOIN paths.
5. **Schema Reduction and Pruning**:
    - Applying Top-K column limits ($K = 8$) and minimum score thresholds ($\theta = 0.15$) to produce the minimal reduced schema dictionary.

Person 3: Full Stack System Design (Database Engine, LLM Pipeline, Server and UI)

Role Summary

Responsible for the **end-to-end full stack architecture**: SQLite database design and PRAGMA introspection, SQL security sandboxing, Google Gemini LLM orchestration and prompt engineering, multithreaded REST API server, and interactive web/CLI user interfaces.

Assigned Code and Assets

- database.py (SQLite initialization, PRAGMA catalog introspection, SQL safety validator, sandboxed execution)
- llm.py (Google Gemini SDK integration, clean SQL regex parser, mock generator)

- prompts.py (Baseline vs Proposed prompt templates)
- server.py (Zero-dependency `ThreadingHTTPServer` REST API)
- static/ (`index.html`, `index.css`, `app.js` with dark theme, real-time comparisons, schema visualizer)
- main.py (Interactive terminal CLI)

Key Technical Concepts and Architecture

1. **Relational Database Design and Dynamic Introspection**:
   - Normalized 4-table `college.db` schema (`Student`, `Course`, `Enrollment`, `Faculty`) with foreign key enforcement.
   - Dynamic schema introspection using `sqlite_master`, `PRAGMA table_info`, and `PRAGMA foreign_key_list`, enabling support for arbitrary custom SQLite databases.
2. **Multi-Layer SQL Security and Sandboxing**:
   - Query validation (validate_sql_safety) rejecting destructive/administrative commands (`ATTACH`, `PRAGMA`, `VACUUM`, `DROP` in read-only mode).
   - Sandboxed execution with transaction rollbacks and parameter validation.
3. **LLM Integration and Prompt Engineering**:
   - Gemini API connection (`google-genai` SDK) with robust SQL extraction (clean_extracted_sql) to strip markdown code blocks and conversational text.
   - Offline pattern-matching mock generator for CI/CD and offline demonstrations.
   - Prompt engineering templates contrasting full-schema baseline vs reduced-schema proposed prompts.
4. **Multithreaded REST API and Frontend Interface**:
   - High-performance Python `ThreadingHTTPServer` exposing `/api/query`, `/api/schema`, `/api/evaluate`, and `/api/custom-db`.
   - Responsive web dashboard featuring side-by-side SQL diff comparisons, live schema reduction counters, execution latency meters, and result tables.

Person 4: Final Metrics Calculation, Quality Control and Trade-off Sweeps

Role Summary

Responsible for the **scientific evaluation benchmark framework, quantitative metric calculations, failure taxonomy classification, hyperparameter sweeps, and test suite quality assurance**.

Assigned Code and Assets

- evaluator.py (`BenchmarkEvaluator`, metrics engine, execution accuracy evaluator, CSV/JSON export)
- sweep_tradeoffs.py (Automated parameter sweeps across Top-K, thresholds, and weights)
- evaluation/questions.json and evaluation/ground_truth.json (15 categorized benchmark test cases)
- tests/test_system.py (Complete 21-test unit test suite)
- `evaluation_results.csv` and `evaluation_results.json`

Key Technical Concepts and Metrics

1. **15-Query Benchmark Dataset Curation**:
   - Comprehensive test queries categorized into *Single-Table Filters*, *Multi-Condition Filters*, *3-Table JOINs*, *Aggregations*, *Group By*, and *Order/Limit*.
   - Ground-truth partitioning of **direct columns** (in SELECT/WHERE) and **structural columns** (foreign keys for JOINs).
2. **Quantitative Evaluation Metrics**:
   - **Schema Precision, Recall, and F1 Score**:

$$\text{Precision} = \frac{|S_{\text{selected}} \cap S_{\text{required}}|}{|S_{\text{selected}}|}, \quad \text{Recall} = \frac{|S_{\text{selected}} \cap S_{\text{required}}|}{|S_{\text{required}}|}, \quad F_1 = 2 \cdot \frac{\text{Prec}}{\text{Preci}}$$

   - **Execution Accuracy (Result-Set Equivalence)**: Executing generated SQL and ground-truth SQL in SQLite and comparing normalized output tuples.
   - **Schema Reduction %**: $\frac{\text{Total Columns} - \text{Reduced Columns}}{\text{Total Columns}} \times 100\%$ ($50\% - 88\%$ reduction).
   - **Prompt Token Savings and Latency Profiling**: Segmenting Schema Linking vs LLM vs DB execution time.
3. **Failure Mode Taxonomy**:
   - Automated categorization into `Schema Linking Miss`, `LLM Syntax Error`, `Join Logic Error`, or `Complete Success`.
4. **Quality Control and Testing**:
   - 21 automated unit tests in tests/test_system.py covering database operations, safety checks, schema linking, evaluation metrics, and mock LLM pipelines.